🚀 Multi-Tenant SaaS Platform — End-to-End Architecture (Implementation → Deployment)

This document consolidates everything built across the full system journey: from backend architecture to frontend UX, microservices, real-time system, and cloud-native deployment.

🧠 1. PRODUCT VISION

We built a **production-grade multi-tenant SaaS platform** inspired by:

- Notion (flexible UX)
- Linear (speed + command palette)
- Slack (multi-workspace + real-time)
- Jira (project/task system)

🏢 2. MULTI-TENANT ARCHITECTURE

Core Concept

Every entity is scoped by:

- organizationId (tenant boundary)

Data Isolation Rule

No query is valid without tenant context:

- organizationId required everywhere

Core Entities

Organization (Tenant)

- id
- name
- users

- projects
- invites

User

- email
- password
- role (RBAC)
- organizationId

Invite System

- email
- token
- role
- organizationId
- expiresAt
- accepted

Project

- name
- organizationId
- ownerId

Task

- title
- status (TODO / IN_PROGRESS / DONE)
- projectId

View (UX abstraction)

- type: KANBAN / LIST / TABLE
- projectId

🔐 3. AUTH SYSTEM

Strategy

- JWT-based authentication
- HTTP-only cookies
- CSRF mitigation (SameSite + token strategy)

JWT Payload

Contains:

- userId
- organizationId
- role

🧩 4. AUTHORIZATION MODEL

RBAC (Role-Based Access Control)

- ADMIN
- MEMBER
- VIEWER

ABAC (Attribute-Based Access Control)

- organizationId filtering
- resource ownership checks
- dynamic rules per entity

🏗️ 5. BACKEND ARCHITECTURE (NESTJS)

Initially built as monolith → evolved into microservices.

⚙️ 6. MICROSERVICES SPLIT

Services

Auth Service

- login / register
- JWT issuing

Project Service

- organizations
- projects
- tasks

- business logic

Realtime Service

- WebSockets
- presence
- event broadcasting

🔄 7. INTER-SERVICE COMMUNICATION

HTTP

- service-to-service calls

Redis Pub/Sub

- event-driven communication
- real-time sync backbone

⚡ 8. REAL-TIME SYSTEM

WebSocket Architecture

- tenant-based rooms (organizationId)
- task updates broadcast
- presence system (users online)

Events

- TASK_UPDATED
- TASK_CREATED
- USER_JOINED

🧠 9. API GATEWAY

Responsibilities

- routing requests to services
- JWT validation (edge layer)
- rate limiting

- request proxying
- central logging entry point

Routes

- /auth → auth service
- /projects → project service
- /ws → realtime service

🧱 10. FRONTEND ARCHITECTURE (NEXT.JS)

App Shell Model

- persistent layout
- dynamic workspace rendering

Layout Structure

- Top Bar
- Sidebar (org + projects)
- Workspace (dynamic)
- Context Panel (optional)

Routing Strategy

- /workspace/:org/:project/:view

🎨 11. MODERN UX SYSTEM

Hybrid Interaction Model

1. Direct UI

- buttons, forms

2. Inline Editing

- edit without navigation

3. Command Palette (Ctrl + K)

- global actions
- fast workflows

📊 12. PROJECT + TASK SYSTEM

Key Idea

Tasks are UI-agnostic data

Views define representation:

- Kanban
- List
- Table

View System

- decouples UI from data
- enables flexible UX

🧲 13. KANBAN SYSTEM (DRAG & DROP)

Implementation

- @dnd-kit
- status-based movement

Features

- drag between columns
- optimistic updates
- real-time sync

⚡ 14. OPTIMISTIC UI

- instant updates

- rollback on failure
- React Query cache mutation

🌐 15. REAL-TIME SYNC (ADVANCED)

Architecture

- WebSocket gateway
- Redis event bus
- tenant-based rooms

Flow

Service → Redis → Realtime service → Clients

🧱 16. RESILIENCE ENGINEERING

Patterns Implemented

Circuit Breaker

- prevents cascading failures

Retry Strategy

- exponential backoff

Timeouts

- prevents hanging requests

Bulkhead

- isolates services

Rate Limiting

- protects system overload

Graceful Degradation

- fallback responses

Idempotency

- prevents duplicate actions

☁️ 17. CLOUD-NATIVE DEPLOYMENT

Infrastructure

- AWS ECS (Fargate)
- RDS PostgreSQL
- ElasticCache Redis
- ECR container registry
- CloudFront CDN

CI/CD Pipeline

1. GitHub push
2. GitHub Actions
3. Docker build
4. Push to ECR
5. ECS deploy
6. Rolling update

Zero Downtime Deployment

- rolling deployments
- no service interruption

🐳 18. DOCKER ARCHITECTURE

Services Containerized

- web (Next.js)
- gateway
- auth service
- project service
- realtime service

Docker Compose (local dev)

- PostgreSQL
- Redis
- all services

🔐 19. SECURITY MODEL

- HTTP-only cookies
- CSRF protection
- JWT validation
- RBAC + ABAC enforcement
- tenant isolation at DB level
- rate limiting at gateway

📈 20. SCALABILITY DESIGN

Horizontal Scaling

- ECS auto scaling
- stateless services

Event-driven architecture

- Redis pub/sub backbone

Future-ready

- Kubernetes migration ready
- microservices isolated

🧠 FINAL SYSTEM SUMMARY

You built a full SaaS platform with:

Backend

- multi-tenant architecture
- microservices
- real-time system

- API gateway

Frontend

- modern app shell UX
- hybrid interaction model
- Kanban + views system
- command palette

Infrastructure

- AWS ECS deployment
- CI/CD pipeline
- Redis + Postgres managed services
- containerized system

Engineering

- resilience patterns
- event-driven design
- scalable architecture
- production-grade deployment

🚀 RESULT

This is now a:

> Fully production-ready, cloud-native, multi-tenant SaaS architecture blueprint with real-time capabilities and microservices scaling design.